



10.2 MOT

MOT特性及价值

- MOT在高性能（查询和事务延迟）、高可扩展性（吞吐量和并发量）甚至在某些情况下成本（高资源利用率）这些方面拥有显著优势。
 - 低延迟（Low Latency）：提供快速的查询和事务响应时间。
 - 高吞吐量（High Throughput）：支持峰值和持续高用户并发。
 - 高资源利用率（High Resource Utilization）：充分利用硬件。
 - 使用了MOT的应用程序可以达到2.5到4倍的吞吐量。例如，在基于Arm/鲲鹏的华为TaiShan服务器和基于英特尔至强的戴尔x86服务器上，执行TPC-C基准测试（交互事务和同步日志）。MOT提供的吞吐率增益在2路服务器上达到2.5倍，4路服务器上达到3.7倍，在4路256核Arm服务器上达到480万tpmC。

关键技术 (1)

- 内存优化数据结构：以实现高并发吞吐量和可预测的低延迟为目标，所有数据和索引都在内存中，不使用中间页缓冲区，并使用持续时间最短的锁。数据结构和所有算法都是专门为内存设计而优化的。
- 免锁事务管理：MOT在保证严格一致性和数据完整性的前提下，采用乐观的策略实现高并发和高吞吐。在事务过程中，MOT不会对正在更新的数据行的任何版本加锁，从而大大降低了一些大内存系统中的争用。事务中的乐观并发控制（Optimistic Concurrency Control, OCC）语句是在没有锁的情况下实现的，所有的数据修改都是在内存中专门用于私有事务的部分（也称为私有事务内存）中进行的。这就意味着在事务过程中，相关数据在私有事务内存中更新，从而实现了无锁读写；而且只有在提交阶段才会短时间加锁。

关键技术 (2)

- 免锁索引：由于内存表的数据和索引完全存储在内存中，因此拥有一个高效的索引数据结构和算法非常重要。MOT索引机制基于最先进的Masstree，这是一种用于多核系统的快速和可扩展的键值（Key Value, KV）存储索引，以B+树的Trie实现。通过这种方式，高并发工作负载在多核服务器上可以获得卓越的性能。同时MOT应用了各种先进的技术以优化性能，如优化锁方法、高速缓存感知和内存预取。
- NUMA-aware的内存管理：由内存控制器处理的，不需要通过使用互连（如英特尔QPI）进行额外的跳转。MOT的智能内存控制模块，为各种内存对象预先分配了内存池，提高了性能，减少了锁，保证了稳定性。事务的内存对象的分配始终是NUMA本地的。本地处理的对象会返回到池中。同时在事务中尽量减少系统内存分配（OS malloc）的使用，避免不必要的锁

关键技术 (3)

- 高效持久性：作为一个基于内存的存储引擎，MOT的持久化设计必须实现各种各样的算法优化，以确保持久化的同时还能达到设计时的速度和吞吐量目标。这些优化包括：
 - 并行日志，所有openGauss磁盘表都支持。
 - 对每个事务做的日志缓冲和无锁事务准备。
 - 增量更新记录优化，即对比原先只记录变化。
 - 除了同步和异步之外，用创新的NUMA感知组提交日志记录。
 - 最先进的数据库检查点（CALC，Checkpointing Asynchronously using Logical Consistency）使内存和计算开销降到最低。

关键技术 (4)

- MOT和openGauss数据库的无缝集成
 - MOT和openGauss数据库的无缝集成：MOT是一个高性能的面向内存优化的存储引擎，已集成在openGauss包中。MOT的主内存引擎和基于磁盘的存储引擎并存，以支持多种应用场景，同时在内部重用数据库辅助服务，如WAL重做日志、复制、检查点和恢复高可用性等。用户可以从基于磁盘的表和MOT的统一部署、配置和访问中受益。根据特定需求，灵活且低成本地选择使用哪种存储引擎。例如，把会导致瓶颈的高度性能敏感数据放入内存中。

MOT应用场景 (1)

- 高吞吐事务处理：这是使用MOT的主要场景，因为它支持海量事务，同时要求单个事务的延迟较低。这类应用的例子有实时决策系统、支付系统、金融工具交易、体育博彩、移动游戏、广告投放等。
- 消除中间层缓存：云计算和移动应用往往会有周期性或峰值的高工作负载。此外，许多应用都有80%以上负载是读负载，并伴有频繁的重复查询。为了满足峰值负载单独要求，以及降低响应延迟提供最佳的用户体验，应用程序通常会部署中间缓存层。这样的附加层增加了开发的复杂性和时间，也增加了运营成本。MOT提供了一个很好的替代方案，通过一致的高性能数据存储来简化应用架构，缩短开发周期，降低CAPEX和OPEX成本。

MOT应用场景 (2)

- 性能瓶颈加速：存在高争用现象的表可以通过使用MOT受益，即使该表是磁盘表。由于延迟更低、竞争和锁更少以及服务器吞吐量能力增加，此类表（除了相关表和在查询和事务中一起引用的表之外）的转换使得性能显著提升。
- 大规模流数据提取：MOT可以满足云端（针对移动、M2M和物联网）、事务处理（Transactional Processing, TP）、分析处理（Analytical Processing, AP）和机器学习（Machine Learning, ML）的大规模流数据的提取要求。MOT尤其擅长持续快速地同时提取来自许多不同来源的大量数据。
- 降低TCO：提高资源利用率和消除中间层可以节省30%到90%的TCO。

MOT使用 (1)

- 授予用户权限
 - 以授予数据库用户对MOT存储引擎的访问权限为例。每个数据库用户仅执行一次，通常在初始配置阶段完成。
 - 要使特定用户能够创建和访问MOT（DDL、DML、SELECT），以下语句只执行一次：
- ```
GRANT USAGE ON FOREIGN SERVER mot_server TO <user>;
```

# MOT使用 (2)

- 创建/删除MOT

- 只有MOT中的创建和删除表语句与openGauss中基于磁盘的表的语句不同。SELECT、DML和DDL的所有其他命令的语法对于MOT表和openGauss基于磁盘的表是一样的。

- 创建MOT：

- 删除名为test的MOT：

```
create FOREIGN table test(x int) [server mot_server];
```

```
drop FOREIGN table test;
```

# MOT使用 (3)

- 为MOT创建索引
  - 支持标准的openGauss创建和删除索引语句。例如：

- create index text\_index1 on test(x) ;

```
create FOREIGN table bmsql_oorder (
o_w_id integer not null,
o_d_id integer not null,
o_id integer not null,
o_c_id integer not null,
o_carrier_id integer,
o_ol_cnt integer,
o_all_local integer,
o_entry_d timestamp,
primary key (o_w_id, o_d_id, o_id)
);
create index bmsql_oorder_index1 on bmsql_oorder(o_w_id, o_d_id, o_c_id, o_id) ;
```